

2026-05-29 - I2C Sensor Integration

Goal

Read data from a real I2C sensor using Zephyr's sensor API, combining Device Tree configuration, driver binding, and the logging skills from previous days.

What you will learn

- How to declare an I2C sensor in the Device Tree overlay
- How to fetch and use a sensor device via `DEVICE_DT_GET`
- How to read channels using `sensor_sample_fetch` and `sensor_channel_get`
- How to use the BME280 (temperature/humidity/pressure) as a worked example

Overview

Zephyr provides a unified sensor API across all supported drivers. Once the sensor is declared in the Device Tree and the correct Kconfig driver is enabled, the application code is the same regardless of the specific chip.

prj.conf

```
CONFIG_I2C=y
CONFIG_SENSOR=y
CONFIG_BME280=y          # replace with your sensor's Kconfig symbol
CONFIG_LOG=y
CONFIG_LOG_DEFAULT_LEVEL=3
```

app.overlay — add sensor to I2C bus

```
&i2c0 {
    bme280@76 {
        compatible = "bosch,bme280";
        reg = <0x76>;          /* I2C address – check your sensor datasheet */
        label = "BME280";
    };
};
```

The @76 suffix and reg value must match the sensor's 7-bit I2C address.

Reading sensor data

```
#include <zephyr/drivers/sensor.h>

const struct device *bme = DEVICE_DT_GET_ANY(bosch_bme280);

void read_sensor(void)
{
    struct sensor_value temp, press, hum;

    sensor_sample_fetch(bme);
    sensor_channel_get(bme, SENSOR_CHAN_AMBIENT_TEMP, &temp);
    sensor_channel_get(bme, SENSOR_CHAN_PRESS, &press);
    sensor_channel_get(bme, SENSOR_CHAN_HUMIDITY, &hum);

    printk("Temp: %d.%02d C  Press: %d.%02d kPa  Hum: %d.%02d %%\n",
           temp.val1, abs(temp.val2) / 10000,
           press.val1, abs(press.val2) / 10000,
           hum.val1, abs(hum.val2) / 10000);
}
```

sensor_value explained

sensor_value is a fixed-point struct: - val1 — integer part - val2 — fractional part in millionths (μ)

So {23, 500000} = 23.5

Common sensor Kconfig symbols

Sensor	Symbol
BME280 temp/hum/press	CONFIG_BME280=y
SHT3xD temp/hum	CONFIG_SHT3XD=y
LIS2DH accelerometer	CONFIG_LIS2DH=y
TMP116 temperature	CONFIG_TMP116=y
ICM-42688 IMU	CONFIG_ICM42688P=y

I2C bus readiness check

Always verify the bus and device are ready before reading:

```
if (!device_is_ready(bme)) {
    LOG_ERR("BME280 not ready – check wiring and I2C address");
    return;
}
```

Why this matters

Almost every IoT product reads from sensors. The Zephyr sensor API means you can swap sensor chips by changing the overlay and Kconfig, with zero changes to your application logic.

Practice tasks

1. Wire a BME280 to I2C pins on the nRF52840 DK (P0.26 = SDA, P0.27 = SCL).
2. Build and flash; confirm temperature readings in the log.
3. Combine with Day 12: encode the sensor reading as CBOR and send it over USB.

Example folder

See `src/2026-05-29_I2C_Sensor_Integration` for the day's code example.

Next topic

You now have the core building blocks. Next steps include BLE, power management, firmware updates via SUIIT/MCUboot, and production hardening.